1. An overview of my BBO approach

- Describe what you are trying to achieve or discover through the BBO capstone project.
- Briefly describe your overall strategy and how your process fits together.

What I am trying to do

I have eight unknown functions. Each one takes a short list of numbers between 0 and 1 and gives back a single number. My job is to find the input that makes that number as large as possible, for all eight.

I cannot see inside any of them. There is no formula and no shortcut. The only way to learn anything is to submit one point and wait a week for the answer, and I get thirteen of those in total per function. So the real skill being tested is not searching hard. It is deciding what is worth measuring.

How my process works

Each week I do the same five things. I add the new result to that function's record. I fit a model that guesses the shape of the function and also says how confident it is at each point. I test that model by hiding one result I already have, refitting without it, and seeing whether the model can predict it. That test score decides how much say the model gets. I then choose the next point. And I write down what I changed and why.

The confidence estimate is the part that matters. A model that only gives a best guess cannot tell me where a measurement would still teach me something. A model that also reports its own uncertainty can.

2. How my strategy has evolved

- Describe the key changes in your approach since your early rounds.
- What influenced these changes - data trends, feedback, model performance or intuition?
- Identify which principles or heuristics now guide your query decisions.

Where I started

I treated all eight problems the same way: one type of model, one standard formula for picking the next point, applied everywhere. It was textbook, and it was too trusting.

What changed

First, I stopped assuming the model was right and started measuring it every week. That weekly score now decides what the model is allowed to do. A model that predicts well picks the next location itself. A middling model is only allowed to rank a handful of small moves I construct around the best point. A model that cannot predict at all picks nothing, and its turns are spent on simple measurement instead. Functions move up and down these tiers as their scores change.

Second, three different model types now compete each week and the best scorer is used. The simpler, more restricted models usually won, which surprised me. With so few data points, a flexible model spends its flexibility fitting noise.

Third, two of the functions produce values spanning a huge range, and nothing worked on them until I modelled the output on a log scale.

Fourth, and most important: when the model disagrees with a direct measurement, the measurement wins. Every one of my biggest gains came from applying that rule.

What drove the changes

Almost entirely model performance and the data itself, usually after something went wrong. Very little of it was intuition. Each change in my notes is tied to a specific result that forced it.

The principles I now work by

- A model gets only as much authority as its test score has earned.
- A direct measurement beats the model's opinion, every time.
- Step sizes come from what the readings show, not from the model's own settings.
- Never write off an input the data has never actually tested.
- After changing a setting, check that the output moved. A change that alters nothing is worse than no change, because it looks like a fix.

3. Patterns, data and insights

- Highlight the most meaningful trends you've observed across your data points or model evaluations.
- Identify which variables or behaviours seem to influence results the most.
- Explain how these observations have shaped your understanding of the search process.

The biggest single factor is scale, not any input

Two functions produce values spanning many orders of magnitude. Until I handled that by working on a log scale, the only thing any model could see was whether a point was near the active region or not. Everything else was invisible underneath it.

Most functions use far fewer inputs than they have

On several, one or two inputs do nearly all the work and the rest barely move the result over the range I sampled. On the most extreme one, every input but a single one has settled at the edge of its allowed range, so the search became one-dimensional.

Optima sit at edges more often than in the middle

I expected peaks in the interior. Several of the best regions turned out to be on the boundary of the allowed space, including one that is simply the far corner.

The strongest signal can point the wrong way

On one function the largest readings by size were all negative. The best region sits just beside that dip, not in it. No standard search method would have found it, because the model correctly saw a low point and correctly avoided it. I had to reframe what I was looking for.

Some functions are noisy and some are exact

On one, the same place gives noticeably different answers each time. Since the score is the best value ever seen, the right response there is to measure the same good spot repeatedly rather than refine it. On others, repeated readings agree exactly.

How this changed my understanding

I stopped thinking of this as searching a space and started thinking of it as identifying the few directions that matter and then measuring carefully along them. Most of the box was never worth visiting.

4. Decision-making and iteration

- Explain how you balance exploration and exploitation.
- Describe one or two examples of strategic decisions. What worked, what didn't, and why?
- Reflect briefly on how you handle uncertainty or adapt when results don't match expectations.

How I balance exploring and exploiting

Deliberately exploratory early, deliberately exploitative late. Early on I searched widely and gave the model a bonus for going where it was uncertain, and I spent whole turns on questions rather than on results: does this input matter at all, how noisy is this function, is there a better region somewhere else. Later I removed those bonuses entirely and simply took the point the model rated highest. The reason is simple. Information is only worth buying if there is a turn left in which to use it.

A decision that worked

On one function, several readings in a row kept rising as I moved along one input, while the model insisted the peak was behind me. I backed the readings instead of the model and pushed further in that direction. It set a record, so I did it again and pushed all the way to the edge. That function is now my best result by a wide margin, and it finished at the corner of the allowed space.

A decision that did not work

Twice I concluded that an input did not matter because the model's fitted settings said so, and twice I was wrong. I spent rounds pushing that input in a direction that made the result worse each time. The cause was subtle: when the model reports no effect, it can actually mean it cannot tell a weak effect from none, given the range it was allowed to consider. Narrowing that range fixed it and improved the model's score at the same time. I now treat 'no effect' as a question rather than an answer.

How I handle uncertainty

I assume the model is wrong until it has earned otherwise, and I size my steps accordingly: when a model is weak, every move is small enough that a bad choice costs little. When something genuinely blocks a decision, I spend a whole turn answering it directly instead of guessing. And when a result contradicts what I expected, I write down what I believed, what came back, and what it ruled out, so that a wrong decision can be traced later rather than quietly repeated.

5. Next steps and reflection

- Outline your planned actions or experiments to improve the model's performance.
- Include two or three sentences connecting your work to the broader ML landscape.
- Add a short statement on how you'd communicate your results to a non-technical audience.

What I would do next

- Vary every input once, early. My clearest mistake was spending round after round refining the inputs I had already measured, while two or three functions were quietly holding most of their remaining value on an input I had never moved at all. A single sweep across every input at the start would have found those gaps far sooner, and it would have been cheap.
- Track predicted against actual every round. My models' test scores looked excellent while one of them was guessing too high on every single prediction it made. The test could not catch that, because it checks how well a model fills gaps between points it already has, not how well it reaches a point it has never seen.
- Average repeated readings on the noisy functions before fitting, so that noise is not mistaken for structure.

How this connects to machine learning more broadly

This is the same problem as tuning a model that takes days to train, screening drug candidates, or designing physical experiments: the thing you want to measure is expensive, opaque, and you only get a few goes. The transferable skill is not running more experiments but deciding which one is worth running, and knowing how far to trust a model that is standing in for reality.

The most general lesson I will carry out of this is that a good validation score does not mean a model is right where you are about to use it. My best-scoring model was also my worst-behaved one, because the score was earned across the whole data set while the decisions were being made in one small corner of it.

How I would explain this to a non-technical audience

We had eight dials to tune. Each test cost a week and we only had thirteen of them per dial, so we could not simply try things. Instead we built a best guess of how each dial affects the result, including how sure we were about that guess, and used it to choose each test. We improved every one of the eight. The most useful thing we learned was that our best guess was least reliable in exactly the place we were looking hardest, so we got into the habit of checking it against direct measurements, and that is where our biggest wins came from.